Artificial Intelligence

BS (CS) Spring 2026

Lab 06 Tasks

Learning Objectives:

1. Local Search Algorithm
2. Hill Climbing
3. Simulated Annealing
4. Local Beam Search

Lab Tasks

Submission Instructions

1. Create one notebook file for all questions with name *ROLLNO_SEC_LAB6* **e.g. i23XXXX_A_LAB06**
2. Submit your .ipynb file with correct name on Google Classroom under your section heading.
3. If you don't follow the above-mentioned submission instruction, you will be marked **zero.**

Plagiarism in the Lab Task will result in **zero** marks in the whole category.

Questions

Q1. In the classic N-Queens problem, the challenge is to place N queens on an N×N chessboard in such a way that no two queens threaten each other. The objective is to find a configuration where no queen can capture another, either horizontally, vertically, or diagonally.

Challenge:

Design a local search algorithm (hill climbing) to optimize the placement of N queens on the chessboard, aiming to find a configuration that minimizes the number of queen conflicts and satisfies the constraints of the N-Queens problem.

Objective:

Develop a local search algorithm that iteratively refines the placement of queens on the chessboard, taking into account the conflicts between queens. The goal is to minimize the number of conflicts and achieve a solution where no queen threatens another.

Requirements:

The local search algorithm should start with an initial placement of queens and iteratively explore neighboring configurations by moving queens to reduce conflicts. The optimization process should consider conflicts along rows, columns, and diagonals and adjust the queen positions accordingly. The algorithm should be designed to work for various board sizes (N×N) and handle scenarios where a solution might not be feasible.

Starter code: Optional use this structure or design your own.

```
import random

def calculate_conflicts(board, n, k):
    """
    TODO: Implement a function to calculate the number of conflicts in
the current board.
    - This function should check:
      1. Column conflicts: More than one queen in a column.
      2. Diagonal conflicts: Queen placed on the main or secondary
diagonals.
    - The function should return the total number of conflicts.
    """
    pass

def get_neighbors(board, n, k):
    """
    TODO: Implement a function to generate neighboring boards.
    - Each neighbor should be created by moving a queen within its row
to a new column.
    - Ensure that the new position is not on a diagonal.
    - Return a list of all possible neighboring boards.
    """
    pass

def hill_climbing_n_queens(board, n, k):
    """
    TODO: Implement the Hill Climbing algorithm to solve the N-Queen
problem.
    - The algorithm should:
      1. Start with the given board.
      2. Generate neighbors and move to the best neighbor (with fewer
conflicts).
      3. Stop when:
         - A solution with zero conflicts is found.
         - No better move is available (local optimum).
    - Return the final board and the number of conflicts.
    """
    pass

def solve_n_queens(n, k, initial_board):
    """
    TODO: Use the above functions to solve the N-Queen problem.
    - Print the initial board and its conflict count.
    - Run the Hill Climbing algorithm.
    - Print the final board and whether a solution was found.
```

```
    """


n   # Board size
k   # Number of queens to place

initial_board = [
    ['.', 'Q', '.', '.', '.', '.'],
    ['.', 'Q', '.', '.', '.', '.'],
    ['.', '.', '.', 'Q', '.', '.'],
    ['.', '.', '.', '.', '.', 'Q'],
    ['.', '.', '.', 'Q', '.', '.'],
    ['.', '.', '.', '.', '.', 'Q']
]

solve_n_queens(n, k, initial_board)
```